

```

import pandas as pd
import zipfile
import os

# Path to the ZIP file in Google Drive
zip_file_path = "/content/drive/MyDrive/Infosys_Datasets_Proj/archive (10).zip"
# Name of the specific CSV file inside the ZIP
csv_file_in_zip = "movies_metadata.csv"
# Temporary directory to extract the CSV
extract_dir = "/tmp/extracted_data"
extracted_csv_path = os.path.join(extract_dir, csv_file_in_zip)

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

try:
    # Open the ZIP file
    with zipfile.ZipFile(zip_file_path, 'r') as zf:
        # Check if the desired CSV file exists within the zip archive
        if csv_file_in_zip in zf.namelist():
            # Extract the specific CSV file
            zf.extract(csv_file_in_zip, extract_dir)
            # Read the extracted CSV file into a DataFrame
            df = pd.read_csv(extracted_csv_path)
            print(f"Successfully loaded '{csv_file_in_zip}' into a DataFrame.")
            print(f"DataFrame head:\n{df.head()}")
        else:
            print(f"Error: '{csv_file_in_zip}' not found inside '{zip_file_path}'")
            print("Files available in the ZIP archive:", zf.namelist())
except FileNotFoundError:
    print(f"Error: The ZIP file '{zip_file_path}' was not found. Please ensure the path is correct and your Google Drive is")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

```

```

Successfully loaded 'movies_metadata.csv' into a DataFrame.
DataFrame head:
  adult  belongs_to_collection  budget  \
0  False  {'id': 10194, 'name': 'Toy Story Collection', ...  30000000
1  False  NaN  65000000
2  False  {'id': 119050, 'name': 'Grumpy Old Men Collect...  0
3  False  NaN  16000000
4  False  {'id': 96871, 'name': 'Father of the Bride Col...  0

      genres  \
0  [{'id': 16, 'name': 'Animation'}, {'id': 35, '...
1  [{'id': 12, 'name': 'Adventure'}, {'id': 14, '...
2  [{'id': 10749, 'name': 'Romance'}, {'id': 35, ...
3  [{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam...
4  [{'id': 35, 'name': 'Comedy'}]

  homepage  id  imdb_id  original_language  \
0  http://toystory.disney.com/toy-story  862  tt0114709  en
1  NaN  8844  tt0113497  en
2  NaN  15602  tt0113228  en
3  NaN  31357  tt0114885  en
4  NaN  11862  tt0113041  en

  original_title  \
0  Toy Story
1  Jumanji
2  Grumpier Old Men
3  Waiting to Exhale
4  Father of the Bride Part II

  overview  ...  release_date  \
0  Led by Woody, Andy's toys live happily in his ...  ...  1995-10-30
1  When siblings Judy and Peter discover an encha...  ...  1995-12-15
2  A family wedding reignites the ancient feud be...  ...  1995-12-22
3  Cheated on, mistreated and stepped on, the wom...  ...  1995-12-22
4  Just when George Banks has recovered from his ...  ...  1995-02-10

  revenue  runtime  spoken_languages  \
0  373554033.0  81.0  [{'iso_639_1': 'en', 'name': 'English'}]
1  262797249.0  104.0  [{'iso_639_1': 'en', 'name': 'English'}, {'iso...
2  0.0  101.0  [{'iso_639_1': 'en', 'name': 'English'}]
3  81452156.0  127.0  [{'iso_639_1': 'en', 'name': 'English'}]
4  76578911.0  106.0  [{'iso_639_1': 'en', 'name': 'English'}]

  status  tagline  \
0  Released  NaN
1  Released  Roll the dice and unleash the excitement!
2  Released  Still Yelling. Still Fighting. Still Ready for...
3  Released  Friends are the people who let you be yourself...
4  Released  Just When His World Is Back To Normal... He's ...

```

	title	video	vote_average	vote_count
0	Toy Story	False	7.7	5415.0
1	Jumanji	False	6.9	2413.0
2	Grumpier Old Men	False	6.5	92.0
3	Waiting to Exhale	False	6.1	34.0
4	Father of the Bride Part II	False	5.7	173.0

```
pip install pandas networkx
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (3.6.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

```
import pandas as pd
import networkx as nx
import json

# Load the movies metadata (the 'df' DataFrame is already loaded from the previous cell)
# Using the already loaded 'df' DataFrame for efficiency.
# If you needed to reload, you would use: movies = pd.read_csv(extracted_csv_path, low_memory=False)
movies = df.copy()

# Inspect to confirm columns
print(movies.columns[:10])

# Create directed graph
G = nx.DiGraph()

# Process first 500 movies for demo
movies_subset = movies.head(500)

for idx, row in movies_subset.iterrows():
    movie_id = row.get("id")
    title = row.get("title", "Unknown Title")
    language = row.get("original_language")

    # Add movie node
    G.add_node(movie_id, label="Movie", title=title)

    # Add language node & relation
    if pd.notna(language):
        G.add_node(language, label="Language")
        G.add_edge(movie_id, language, relation="IN_LANGUAGE")

    # Genres: stored as stringified JSON; parse if present
    genre_info = row.get("genres")
    if pd.notna(genre_info):
        try:
            genres = json.loads(genre_info.replace("'", '"'))
            for g in genres:
                genre_name = g.get("name")
                if genre_name:
                    G.add_node(genre_name, label="Genre")
                    G.add_edge(movie_id, genre_name, relation="HAS_GENRE")
        except:
            pass

    # Production companies similarly
    prod_info = row.get("production_companies")
    if pd.notna(prod_info):
        try:
            comps = json.loads(prod_info.replace("'", '"'))
            for comp in comps:
                comp_name = comp.get("name")
                if comp_name:
                    G.add_node(comp_name, label="ProductionCompany")
                    G.add_edge(movie_id, comp_name, relation="MADE_BY")
        except:
            pass

print("Nodes:", len(G.nodes()), "Edges:", len(G.edges()))

Index(['adult', 'belongs_to_collection', 'budget', 'genres', 'homepage', 'id',
      'imdb_id', 'original_language', 'original_title', 'overview'],
      dtype='object')
Nodes: 1020 Edges: 2754
```

```
genres = [n for n, d in G.nodes(data=True) if d.get("label") == "Genre"]
print("Genres in graph:", genres)
```

Genres in graph: ['Animation', 'Comedy', 'Family', 'Adventure', 'Fantasy', 'Romance', 'Drama', 'Action', 'Crime', 'Thriller']

```
movie_to_query = "The Matrix" # Example title
movie_nodes = [(n,d) for n,d in G.nodes(data=True) if d.get("label")=="Movie" and d.get("title")==movie_to_query]

if movie_nodes:
    node_id = movie_nodes[0][0]
    for s,t,d in G.out_edges(node_id, data=True):
        print(f"{movie_to_query} -[{d['relation']}]>-> {t}")
else:
    print("Movie not in subset.")
```

Movie not in subset.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45466 entries, 0 to 45465
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   adult                  45466 non-null  object
1   belongs_to_collection  4494 non-null   object
2   budget                 45466 non-null  object
3   genres                 45466 non-null  object
4   homepage               7782 non-null   object
5   id                     45466 non-null  object
6   imdb_id                45449 non-null  object
7   original_language      45455 non-null  object
8   original_title         45466 non-null  object
9   overview               44512 non-null  object
10  popularity             45461 non-null  object
11  poster_path            45080 non-null  object
12  production_companies   45463 non-null  object
13  production_countries   45463 non-null  object
14  release_date           45379 non-null  object
15  revenue                 45460 non-null  float64
16  runtime                 45203 non-null  float64
17  spoken_languages       45460 non-null  object
18  status                 45379 non-null  object
19  tagline                 20412 non-null  object
20  title                   45460 non-null  object
21  video                  45460 non-null  object
22  vote_average           45460 non-null  float64
23  vote_count             45460 non-null  float64
dtypes: float64(4), object(20)
memory usage: 8.3+ MB
```

```
movie_to_query = "Clueless" # Example title
movie_nodes = [(n,d) for n,d in G.nodes(data=True) if d.get("label")=="Movie" and d.get("title")==movie_to_query]

if movie_nodes:
    node_id = movie_nodes[0][0]
    for s,t,d in G.out_edges(node_id, data=True):
        print(f"{movie_to_query} -[{d['relation']}]>-> {t}")
else:
    print("Movie not in subset.")
```

```
Clueless -[IN_LANGUAGE]>-> en
Clueless -[HAS_GENRE]>-> Comedy
Clueless -[HAS_GENRE]>-> Drama
Clueless -[HAS_GENRE]>-> Romance
Clueless -[MADE_BY]>-> Paramount Pictures
```

#Install Neo4j Python Driver in Colab

pip install neo4j pandas

```
Collecting neo4j
  Downloading neo4j-6.1.0-py3-none-any.whl.metadata (5.3 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: pytz in /usr/local/lib/python3.12/dist-packages (from neo4j) (2025.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
Downloading neo4j-6.1.0-py3-none-any.whl (325 kB)
----- 325.3/325.3 kB 10.0 MB/s eta 0:00:00
Installing collected packages: neo4j
Successfully installed neo4j-6.1.0
```

```

import pandas as pd
import zipfile
import os

# Reusing variables from the initial extraction in cell b87T10GGfF_N
zip_file_path = "/content/drive/MyDrive/Infosys_Datasets_Proj/archive (10).zip"
extract_dir = "/tmp/extracted_data"
# Ensure the extraction directory exists (it should from the previous cell, but good practice)
os.makedirs(extract_dir, exist_ok=True)

# 'movies_metadata.csv' is already loaded into `df` from the previous cell.
# We can assign it to `movies` for consistency with the cell's original intent.
movies = df.copy() # Use a copy if you intend to modify 'movies' independently later.

# Define the other CSV files to extract and load
csv_files_to_load = ["credits.csv", "keywords.csv"]
loaded_dataframes = {}

try:
    with zipfile.ZipFile(zip_file_path, 'r') as zf:
        zip_contents = zf.namelist()
        for csv_file_name in csv_files_to_load:
            if csv_file_name in zip_contents:
                extracted_path = os.path.join(extract_dir, csv_file_name)
                zf.extract(csv_file_name, extract_dir)
                loaded_dataframes[csv_file_name.replace('.csv', '')] = pd.read_csv(extracted_path)
                print(f"Successfully loaded '{csv_file_name}'.")
            else:
                print(f"Error: '{csv_file_name}' not found inside '{zip_file_path}'.")
                print("Files available in the ZIP archive:", zip_contents)

    credits = loaded_dataframes.get('credits')
    keywords = loaded_dataframes.get('keywords')

    # Print shapes if all dataframes were successfully loaded
    if credits is not None and keywords is not None:
        print(movies.shape, credits.shape, keywords.shape)
    else:
        print("Some dataframes could not be loaded.")

except FileNotFoundError:
    print(f"Error: The ZIP file '{zip_file_path}' was not found. Please ensure the path is correct and your Google Drive is")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

```

```

Successfully loaded 'credits.csv'.
Successfully loaded 'keywords.csv'.
(45466, 24) (45476, 3) (46419, 2)

```

#Clean & Select Required Columns

```

movies = movies[['id', 'title', 'original_language', 'release_date', 'genres']]
credits = credits[['id', 'cast', 'crew']]
keywords = keywords[['id', 'keywords']]

movies = movies.dropna(subset=['id', 'title'])

```

#Connect to Neo4j from Colab

```

from neo4j import GraphDatabase

URI = "neo4j+s://e27135de.databases.neo4j.io"
AUTH = ("neo4j", "WkmpufxtnWTdPav7XT1Ae9KD-_5H30FodCux9H1QLYY")

driver = GraphDatabase.driver(URI, auth=AUTH)

```

```

def insert_movie(tx, movie_id, title, lang, date):
    tx.run("""
        MERGE (m:Movie {id:$id})
        SET m.title=$title,
            m.language=$lang,
            m.release_date=$date
        """, id=movie_id, title=title, lang=lang, date=date)

with driver.session() as session:
    for _, row in movies.head(1000).iterrows(): # demo subset
        session.execute_write(
            insert_movie,

```

```
        row['id'], row['title'],
        row['original_language'], row['release_date']
    )
```

#Version

```
import neo4j
print(neo4j.__version__)
```

6.1.0

```
movies_clean = movies.copy()

# Convert id to numeric and drop bad rows
movies_clean['id'] = pd.to_numeric(movies_clean['id'], errors='coerce')
movies_clean = movies_clean.dropna(subset=['id', 'title'])

# Convert id to int
movies_clean['id'] = movies_clean['id'].astype(int)

movies_clean = movies_clean.head(1000)
print(movies_clean.shape)
```

(1000, 5)

```
from neo4j import GraphDatabase

URI = "neo4j+s://e27135de.databases.neo4j.io"
AUTH = ("neo4j", "WkmpufxtnWTdPav7XT1Ae9KD-_5H30FodCux9H1QLYY")

driver = GraphDatabase.driver(URI, auth=AUTH)
```

#Insert Movies into Neo4j

```
def insert_movie(tx, movie_id, title, lang, date):
    tx.run(
        """
        MERGE (m:Movie {id: $id})
        SET m.title = $title,
            m.language = $lang,
            m.release_date = $date
        """,
        id=movie_id,
        title=title,
        lang=lang,
        date=date
    )
```

```
with driver.session() as session:
    for _, row in movies_clean.iterrows():
        session.execute_write(
            insert_movie,
            row['id'],
            row['title'],
            row['original_language'],
            row['release_date']
        )

print("✅ Movies inserted successfully")
```

✅ Movies inserted successfully

```
with driver.session() as session:
    result = session.run("RETURN 1 AS test")
    print(result.single()["test"])
```

1

```
with driver.session() as session:
    session.run("""
        MERGE (m:Movie {id: 999999})
        SET m.title = 'Test Movie'
    """)
    print("Single insert OK")
```

Single insert OK

```

count = 0

with driver.session() as session:
    for _, row in movies_clean.head(50).iterrows(): # START SMALL
        session.run(
            """
            MERGE (m:Movie {id: $id})
            SET m.title = $title,
                m.language = $lang,
                m.release_date = $date
            """,
            id=int(row['id']),
            title=str(row['title']),
            lang=str(row['original_language']),
            date=str(row['release_date'])
        )
        count += 1

print(f"✅ Inserted {count} movies")

```

✅ Inserted 50 movies

```

with driver.session() as session:
    for _, row in movies_clean.head(100).iterrows():
        session.run(
            """
            MERGE (m:Movie {id: $id})
            SET m.title = $title,
                m.language = $lang,
                m.release_date = $date
            """,
            id=int(row['id']),
            title=str(row['title']),
            lang=str(row['original_language']),
            date=str(row['release_date'])
        )

print("✅ Movies inserted successfully")

```

✅ Movies inserted successfully

```

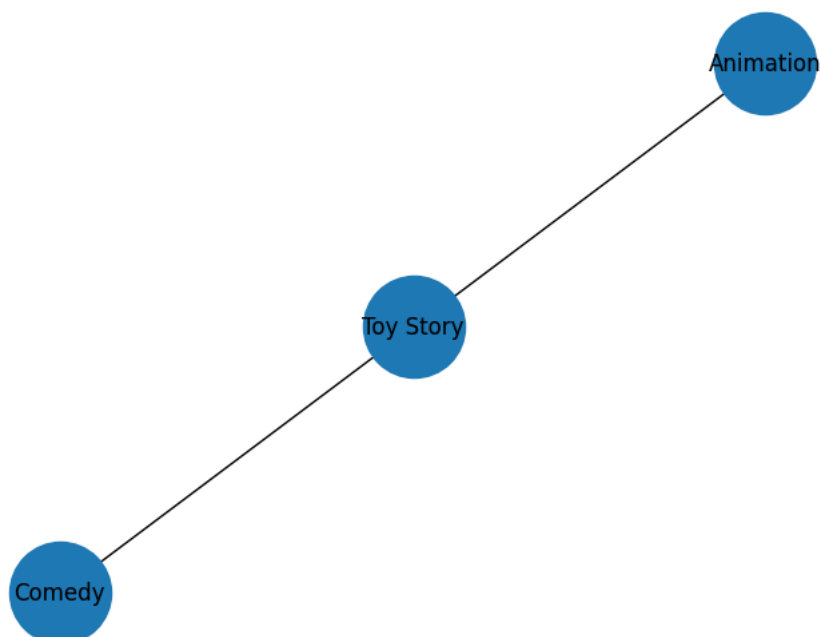
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

G.add_edge("Toy Story", "Animation")
G.add_edge("Toy Story", "Comedy")

nx.draw(G, with_labels=True, node_size=3000)
plt.show()

```



```

with driver.session() as session:
    result = session.run("""
    CALL apoc.export.csv.query(
        "MATCH (m:Movie)-[:HAS_GENRE]->(g:Genre)
        RETURN m.title AS movie, g.name AS genre",
        null,
        {stream:true}
    )
    """)
    record = result.single()
    csv_data = record["data"]

with open("movies_genres.csv", "w", encoding="utf-8") as f:
    f.write(csv_data)

```

```

query = """
MATCH (m:Movie)-[:HAS_GENRE]->(g:Genre)
RETURN m.title AS movie, g.name AS genre
LIMIT 500
"""

with driver.session() as session:
    result = session.run(query)
    df = pd.DataFrame([r.data() for r in result])

df.to_csv("movies_genres.csv", index=False)
df.head()

```

	movie	genre
0	Toy Story	Animation
1	Balto	Animation
2	Pocahontas	Animation
3	A Goofy Movie	Animation
4	Gumby: The Movie	Animation

```

with driver.session() as session:
    session.run("CREATE CONSTRAINT movie_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.id IS UNIQUE")
    session.run("CREATE CONSTRAINT person_name IF NOT EXISTS FOR (p:Person) REQUIRE p.name IS UNIQUE")

```

```

movies_small = movies.head(200) # for demo
movie_data = movies_small[["id", "title", "release_date"]].to_dict("records")

```

```

def create_movies_batch(tx, movies):
    tx.run("""
    UNWIND $movies AS movie
    MERGE (m:Movie {id: movie.id})
    SET m.title = movie.title,
        m.release_date = movie.release_date
    """, movies=movies)

with driver.session() as session:
    session.execute_write(create_movies_batch, movie_data)

print("✅ Movies inserted successfully")

```

✅ Movies inserted successfully

```

from tqdm import tqdm

batch_size = 50

for i in tqdm(range(0, len(movie_data), batch_size)):
    batch = movie_data[i:i+batch_size]
    with driver.session() as session:
        session.execute_write(create_movies_batch, batch)

```

100%|██████████| 4/4 [00:02<00:00, 1.83it/s]

```

import networkx as nx
import matplotlib.pyplot as plt

# Create graph
G = nx.Graph()

# Movies → Genres

```

```

G.add_edge("Toy Story", "Animation")
G.add_edge("Toy Story", "Comedy")

G.add_edge("The Matrix", "Action")
G.add_edge("The Matrix", "Sci-Fi")

G.add_edge("Inception", "Sci-Fi")
G.add_edge("Inception", "Thriller")

# Movies → Actors
G.add_edge("Toy Story", "Tom Hanks")
G.add_edge("The Matrix", "Keanu Reeves")
G.add_edge("Inception", "Leonardo DiCaprio")

# Define node colors by type
node_colors = []
for node in G.nodes():
    if node in ["Toy Story", "The Matrix", "Inception"]:
        node_colors.append("lightblue")    # Movies
    elif node in ["Animation", "Comedy", "Action", "Sci-Fi", "Thriller"]:
        node_colors.append("lightgreen")    # Genres
    else:
        node_colors.append("lightcoral")    # Actors

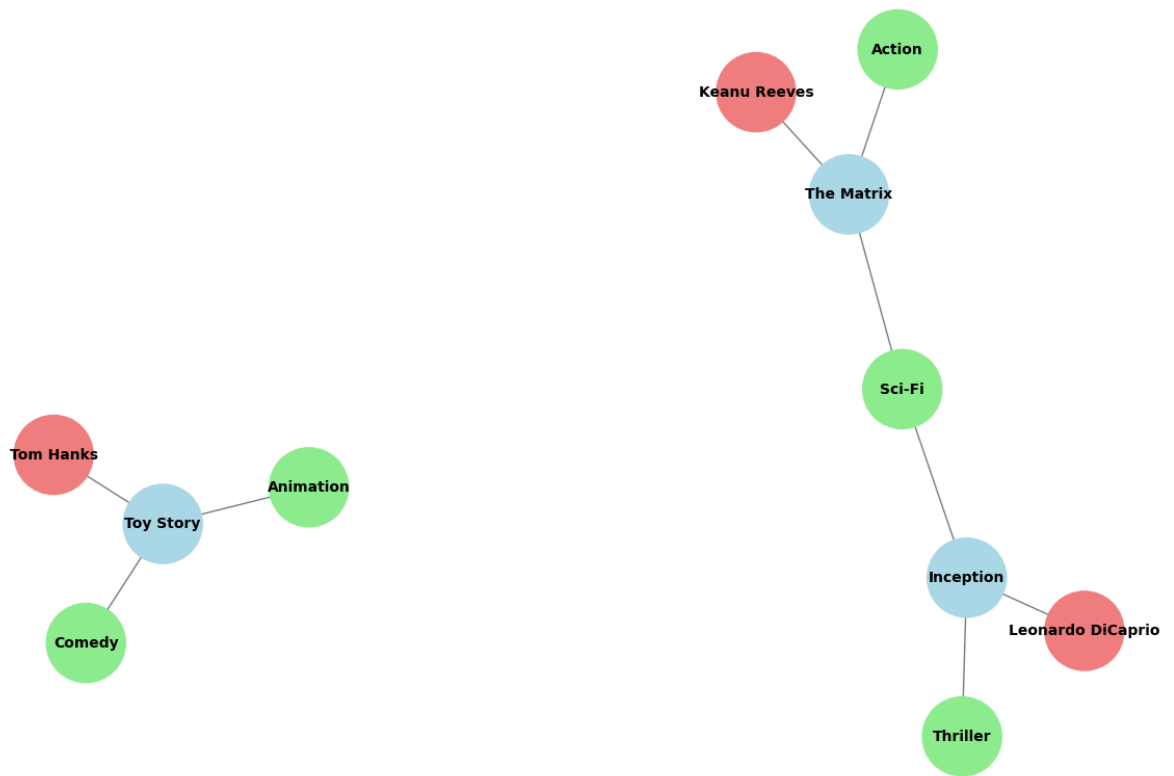
# Draw graph
plt.figure(figsize=(12, 8))
pos = nx.spring_layout(G, seed=42)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=node_colors,
    node_size=3000,
    font_size=10,
    font_weight="bold",
    edge_color="gray"
)

plt.title("Movie Knowledge Graph (Movies-Genres-Actors)", fontsize=14)
plt.show()

```


Movie Knowledge Graph (Movies-Genres-Actors)



```
import networkx as nx
import matplotlib.pyplot as plt

# Create graph
G = nx.Graph()

# -----
# Add edges with semantics
# -----
edges = [
    ("Toy Story", "Animation", "HAS_GENRE"),
    ("Toy Story", "Comedy", "HAS_GENRE"),
    ("Toy Story", "Tom Hanks", "ACTED_IN"),

    ("The Matrix", "Action", "HAS_GENRE"),
    ("The Matrix", "Sci-Fi", "HAS_GENRE"),
    ("The Matrix", "Keanu Reeves", "ACTED_IN"),

    ("Inception", "Sci-Fi", "HAS_GENRE"),
    ("Inception", "Thriller", "HAS_GENRE"),
    ("Inception", "Leonardo DiCaprio", "ACTED_IN")
]

for u, v, rel in edges:
    G.add_edge(u, v, label=rel)

# -----
# Node classification
# -----
movies = ["Toy Story", "The Matrix", "Inception"]
genres = ["Animation", "Comedy", "Action", "Sci-Fi", "Thriller"]

node_colors = []
for node in G.nodes():
    if node in movies:
        node_colors.append("#8ecae6")    # Movie - blue
```

```

        elif node in genres:
            node_colors.append("#90db97")    # Genre - green
        else:
            node_colors.append("#f4a261")    # Actor - orange

# -----
# Draw graph
# -----
plt.figure(figsize=(14, 10))
pos = nx.spring_layout(G, seed=42)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color=node_colors,
    node_size=3500,
    font_size=11,
    font_weight="bold",
    edge_color="gray"
)

# Edge labels
edge_labels = nx.get_edge_attributes(G, "label")
nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels,
    font_size=9
)

# -----
# Legend
# -----
from matplotlib.patches import Patch

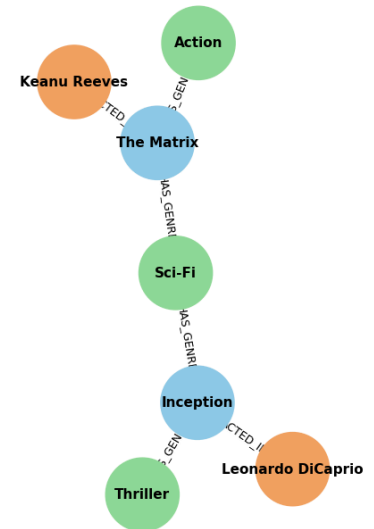
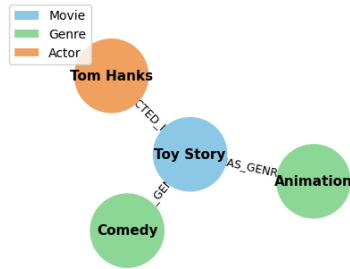
legend_elements = [
    Patch(facecolor="#8ecae6", label="Movie"),
    Patch(facecolor="#90db97", label="Genre"),
    Patch(facecolor="#f4a261", label="Actor")
]

plt.legend(handles=legend_elements, loc="upper left")

plt.title("Movie Knowledge Graph (Entities & Relationships)", fontsize=16)
plt.axis("off")
plt.show()

```

Movie Knowledge Graph (Entities & Relationships)



#Milestone_3

```

import pandas as pd
import numpy as np
# Install faiss-cpu if not already installed
try:
    import faiss
except ImportError:
    !pip install faiss-cpu
    import faiss
import pickle
from sentence_transformers import SentenceTransformer

# Load KG-linked text
df = pd.read_csv(extracted_csv_path)

# Use 'overview' for text and 'id' for entity_ids from movies_metadata.csv
texts = df["overview"].dropna().tolist()
entity_ids = df.loc[df["overview"].notna(), "id"].tolist()

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Generate embeddings
embeddings = model.encode(texts, show_progress_bar=True)
embeddings = np.array(embeddings)

print("Embedding shape:", embeddings.shape)
  
```

```

# Build FAISS index (Flat L2)
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)

# Persist index
faiss.write_index(index, "kg_embeddings.faiss")

# Save entity mapping
with open("entity_map.pkl", "wb") as f:
    pickle.dump(list(zip(entity_ids, texts)), f)

print("FAISS index built successfully")

```

```

/tmp/ipython-input-202244622.py:13: DtypeWarning: Columns (10) have mixed types. Specify dtype option on import or set low_memory=True.
df = pd.read_csv(extracted_csv_path)

Loading weights: 100% 103/103 [00:00<00:00, 437.87it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | 
-----+-----+
embeddings.position_ids | UNEXPECTED | 

Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Batches: 100% 1391/1391 [33:51<00:00, 4.93it/s]
Embedding shape: (44512, 384)
FAISS index built successfully

```

```

import pandas as pd
import numpy as np
import faiss
import pickle
from sentence_transformers import SentenceTransformer

# Load dataset
movies = pd.read_csv(extracted_csv_path, low_memory=False)
movies = movies[["id", "title", "overview"]].dropna().head(500)

texts = movies["overview"].tolist()

# Embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Generate embeddings
embeddings = model.encode(texts, show_progress_bar=True)
embeddings = np.array(embeddings)

# Build FAISS index
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)

# Save FAISS index
faiss.write_index(index, "movie_index.faiss")

# Save metadata
with open("documents.pkl", "wb") as f:
    pickle.dump(movies.to_dict("records"), f)

print("✅ Milestone 3: FAISS index + metadata saved")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 15.1kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 4.86kB/s]

README.md: 10.5k/? [00:00<00:00, 406kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 2.28kB/s]

config.json: 100% 612/612 [00:00<00:00, 26.1kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 67.7MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 405.54it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 17.7kB/s]

vocab.txt: 232k/? [00:00<00:00, 7.23MB/s]

tokenizer.json: 466k/? [00:00<00:00, 18.7MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 10.0kB/s]

config.json: 100% 190/190 [00:00<00:00, 14.9kB/s]

Batches: 100% 16/16 [00:21<00:00, 1.91it/s]
✅ Milestone 3: FAISS index + metadata saved

```

```

from fastapi import FastAPI
from pydantic import BaseModel
import faiss
import pickle
from sentence_transformers import SentenceTransformer
from neo4j import GraphDatabase

# Load FAISS + metadata
index = faiss.read_index("movie_index.faiss")

with open("documents.pkl", "rb") as f:
    documents = pickle.load(f)

model = SentenceTransformer("all-MiniLM-L6-v2")

# Neo4j connection (from Milestone 2)
driver = GraphDatabase.driver(
    "bolt://localhost:7687",
    auth=("neo4j", "password")
)

app = FastAPI(title="Milestone 3 - Semantic Search & Graph-Aware RAG")

class Query(BaseModel):
    query: str
    top_k: int = 5

@app.get("/")
def health():
    return {"status": "Milestone 3 backend running"}

# 🔍 Semantic Search
@app.post("/semantic-search")
def semantic_search(payload: Query):
    query_vector = model.encode([payload.query])
    distances, indices = index.search(query_vector, payload.top_k)

    results = []
    for idx in indices[0]:
        results.append({
            "id": documents[idx]["id"],
            "title": documents[idx]["title"],
            "overview": documents[idx]["overview"]

```

